10. 페이로드 빌더 (Payload Builder) 분석

개요

페이로드 빌더는 Reth에서 새로운 블록을 생성하는 핵심 컴포넌트입니다.
`EthereumPayloadBuilder` 는 트랜잭션 풀에서 트랜잭션을 선택하고, 이를 실행하여 유효한 블록 페이로드를 구성하는 역할을 담당합니다. 이 컴포넌트는 MEV(Maximal Extractable Value) 최적화와 트랜잭션 선택 알고리즘을 통해 블록 생성 과정을 효율적으로 관리합니다.

핵심 구조체 및 트레이트

EthereumPayloadBuilder

```
#[derive(Debug, Clone, PartialEq, Eq)]

pub  struct EthereumPayloadBuilder<Pool, Client, EvmConfig = EthEvmConfig> {

/// 노드 상태에 접근하는 클라이언트

client: Client,

/// 트랜잭션 풀

pool: Pool,

/// EVM 생성을 담당하는 타입

evm_config: EvmConfig,

/// 페이로드 빌더 설정

builder_config: EthereumBuilderConfig,
```

```
}
```

EthereumBuilderConfig

```
#[derive(PartialEq, Eq, Clone, Debug)]
pub struct EthereumBuilderConfig {
/// 목표 가스 한도
pub desired_gas_limit: u64,
/// 페이로드가 없을 때 첫 번째 페이로드 빌드를 기다릴지 여부
pub await_payload_on_missing: bool,
}
```

PayloadBuilder 트레이트

```
impl<Pool, Client, EvmConfig> PayloadBuilder for EthereumPayloadBuilder<Pool, Cl
where
EvmConfig: ConfigureEvm<Primitives = EthPrimitives, NextBlockEnvCtx = NextBlockE
Client: StateProviderFactory + ChainSpecProvider<ChainSpec: EthereumHardforks> +
Pool: TransactionPool<Transaction: PoolTransaction<Consensus = TransactionSigned
{
type Attributes = EthPayloadBuilderAttributes;
type BuiltPayload = EthBuiltPayload;
```

```
fn try_build(&self, args: BuildArguments<EthPayloadBuilderAttributes, EthBuiltP
-> Result<BuildOutcome<EthBuiltPayload>, PayloadBuilderError>;
}
```

블록 생성 과정

1. 초기화 단계

```
// 상태 프로바이더 생성
let state_provider = client.state_by_block_hash(parent_header.hash())?;
let state = StateProviderDatabase::new(&state_provider);
let mut db = State::builder()
.with_database(cached_reads.as_db_mut(state))
.with_bundle_update()
.build();

// EVM 빌더 생성
let mut builder = evm_config.builder_for_next_block(
&mut db,
&parent_header,
NextBlockEnvAttributes {
timestamp: attributes.timestamp(),
suggested_fee_recipient: attributes.suggested_fee_recipient(),
```

```
prev_randao: attributes.prev_randao(),
gas_limit: builder_config.gas_limit(parent_header.gas_limit),
parent_beacon_block_root: attributes.parent_beacon_block_root(),
withdrawals: Some(attributes.withdrawals().clone()),
},
)?;
```

2. 트랜잭션 선택 및 실행

```
// 최적 트랜잭션 이터레이터 생성
let mut best_txs = best_txs(BestTransactionsAttributes::new(
base_fee,
builder.evm_mut().block().blob_gasprice().map(|gasprice| gasprice as u64),
));

while let Some(pool_tx) = best_txs.next() {
// 가스 한도 검증
if cumulative_gas_used + pool_tx.gas_limit() > block_gas_limit {
best_txs.mark_invalid(&pool_tx, InvalidPoolTransactionError::ExceedsGasLimit);
continue;
}

// 트랜잭션 실행
```

```
let  gas_used = match  builder.execute_transaction(tx.clone()) {
Ok(gas_used) => gas_used,
Err(err) => {
// 오류 처리 및 무효 트랜잭션 마킹
continue;
}
};

// 수수료 계산 및 누적
let  miner_fee = tx.effective_tip_per_gas(base_fee)
.expect("fee is always valid; execution succeeded");
total_fees += U256::from(miner_fee) * U256::from(gas_used);
cumulative_gas_used += gas_used;
}
```

3. 블록 완성

```
// 블록 빌드 완료
let BlockBuilderOutcome { execution_result, block, .. } = builder.finish(&state_

// 요청 처리 (Prague 하드포크 이후)
let  requests = chain_spec
.is_prague_active_at_timestamp(attributes.timestamp)
```

```
.then_some(execution_result.requests);


// 최종 페이로드 생성
let  payload = EthBuiltPayload::new(attributes.id, sealed_block, total_fees, req
.with_sidecars(blob_sidecars);
```

트랜잭션 선택 알고리즘

우선순위 기반 선택

Reth는 `CoinbaseTipOrdering` 을 기본 트랜잭션 정렬 방식으로 사용합니다:

```
impl<T> TransactionOrdering for CoinbaseTipOrdering<T>
where
T: PoolTransaction + 'static,
{
type PriorityValue = u128;
type Transaction = T;


fn  priority(&self, transaction: &Self::Transaction, base_fee: u64) -> Priority<
// effective_tip_per_gas = min(max_priority_fee_per_gas, max_fee_per_gas - base_
transaction.effective_tip_per_gas(base_fee).into()
}
```

```
}
```

BestTransactions 이터레이터

트랜잭션 선택은 `BestTransactions` 이터레이터를 통해 이루어집니다:

```
pub struct BestTransactions<T: TransactionOrdering> {
/// 모든 대기 중인 트랜잭션의 복사본
pub(crate) all: BTreeMap<TransactionId, PendingTransaction<T>>,
/// 즉시 실행 가능한 독립적인 트랜잭션들
pub(crate) independent: BTreeSet<PendingTransaction<T>>,
/// 무효한 트랜잭션 추적
pub(crate) invalid: HashSet<SenderId>,
/// 새로운 대기 트랜잭션 수신기
pub(crate) new_transaction_receiver: Option<Receiver<PendingTransaction<T>>>,
}
```

수수료 기반 필터링

`BestTransactionsWithFees` 는 base fee와 blob fee를 만족하는 트랜잭션만 선택합니다:

```
impl<T: TransactionOrdering> Iterator for BestTransactionsWithFees<T> {
fn next(&mut self) -> Option<Self::Item> {
loop {
let best = Iterator::next(&mut self.best)?;
```

```
// base fee와 blob fee 조건 확인
if best.transaction.max_fee_per_gas() >= self.base_fee as u128 &&
best.transaction.max_fee_per_blob_gas()
.is_none_or(|fee| fee >= self.base_fee_per_blob_gas as u128)
{
return Some(best);
}
// 조건을 만족하지 않으면 무효 처리
self.mark_invalid(&best, InvalidPoolTransactionError::Underpriced);
}
}
}
```

MEV 관련 기능 및 최적화

1. 효과적인 팁 계산

MEV 최적화의 핵심은 각 트랜잭션의 실제 수익성을 정확히 계산하는 것입니다:

```
// 마이너가 받는 실제 수수료 계산
let miner_fee = tx.effective_tip_per_gas(base_fee)
.expect("fee is always valid; execution succeeded");
total_fees += U256::from(miner_fee) * U256::from(gas_used);
```

2. 우선순위 발신자 지원

특정 발신자의 트랜잭션을 우선 처리하는 기능:

```
pub struct BestTransactionsWithPrioritizedSenders<I: Iterator> {
/// 내부 이터레이터
inner: I,
/// 우선순위를 받을 발신자 집합
prioritized_senders: HashSet<Address>,
/// 우선순위 트랜잭션의 최대 총 가스 한도
max_prioritized_gas: u64,
/// 우선순위가 아닌 트랜잭션 버퍼
buffer: VecDeque<I::Item>,
/// 우선순위 트랜잭션의 총 가스 추적
prioritized_gas: u64,
}
```

3. 동적 가스 한도 조정

블록의 가스 한도는 부모 블록을 기반으로 동적으로 조정됩니다:

```
pub fn calculate_block_gas_limit(parent_gas_limit: u64, desired_gas_limit: u64
let delta = (parent_gas_limit / GAS_LIMIT_BOUND_DIVISOR).saturating_sub(1);
let min_gas_limit = parent_gas_limit - delta;
let max_gas_limit = parent_gas_limit + delta;
```

```
desired_gas_limit.clamp(min_gas_limit, max_gas_limit)

}
```

4. Blob 트랜잭션 최적화

EIP-4844 blob 트랜잭션에 대한 특별한 처리:

```
// Blob 개수 제한 확인

if  block_blob_count + tx_blob_count > max_blob_count {

best_txs.mark_invalid(&pool_tx, InvalidPoolTransactionError::Eip4844(

Eip4844PoolTransactionError::TooManyEip4844Blobs {

have: block_blob_count + tx_blob_count,

permitted: max_blob_count,

},

));

continue;

}


// Blob 최대 개수에 도달하면 blob 트랜잭션 건너뛰기

if  block_blob_count == max_blob_count {

best_txs.skip_blobs();

}
```

성능 최적화 방식

1. 캐시된 읽기 활용

```
let mut db = State::builder()
.with_database(cached_reads.as_db_mut(state))
.with_bundle_update()
.build();
```

2. 조기 종료 조건

```
// 더 나은 페이로드가 없으면 빌드 중단
if !is_better_payload(best_payload.as_ref(), total_fees) {
return Ok(BuildOutcome::Aborted { fees: total_fees, cached_reads });
}

// 작업이 취소되면 조기 종료
if cancel.is_cancelled() {
return Ok(BuildOutcome::Cancelled);
}
```

3. 블록 크기 제한 (Osaka 하드포크)

```
let estimated_block_size_with_tx = block_transactions_rlp_length +
tx.inner().length() +
```

```
attributes.withdrawals().length() +
1024; // 블록 헤더 오버헤드


if  is_osaka && estimated_block_size_with_tx > MAX_RLP_BLOCK_SIZE {
best_txs.mark_invalid(&pool_tx, InvalidPoolTransactionError::OversizedData);
continue;
}
```

커스터마이징 및 확장

커스텀 페이로드 빌더 구현

```
#[derive(Debug, Clone, Copy, Default)]
pub  struct CustomPayloadBuilder;


impl<Node, Pool> PayloadServiceBuilder<Node, Pool, EthEvmConfig> for CustomPaylo
async  fn  spawn_payload_builder_service(
self,
ctx: &BuilderContext<Node>,
pool: Pool,
evm_config: EthEvmConfig,
) -> eyre::Result<PayloadBuilderHandle<_>> {
// 커스텀 페이로드 빌더 로직
```

```
let payload_builder = reth_ethereum_payload_builder::EthereumPayloadBuilder::ne
ctx.provider().clone(),
pool,
evm_config,
EthereumBuilderConfig::new(),
);
// 서비스 생성 및 반환
// ...
}
}
```

설정 옵션

```
impl EthereumBuilderConfig {
pub const fn new() -> Self {
Self {
desired_gas_limit: ETHEREUM_BLOCK_GAS_LIMIT_30M,
await_payload_on_missing: true
}
}

pub const fn with_gas_limit(mut self, desired_gas_limit: u64) -> Self {
self.desired_gas_limit = desired_gas_limit;
```

```
self
}

pub const fn with_await_payload_on_missing(mut self, await_payload_on_missin
self.await_payload_on_missing = await_payload_on_missing;
self
}
}
```

관련 컴포넌트

의존성 관계

- **트랜잭션 풀**: 실행할 트랜잭션 제공
- **실행 엔진**: 트랜잭션 실행 및 상태 업데이트
- **스토리지 시스템**: 상태 데이터 접근
- **합의 메커니즘**: 블록 검증 및 확정

상호작용 방식

1. **Engine API**를 통해 합의 레이어로부터 페이로드 빌드 요청 수신
2. **트랜잭션 풀**에서 최적의 트랜잭션 선택
3. **실행 엔진**을 통해 트랜잭션 실행
4. **스토리지 시스템**에서 상태 업데이트

5. 완성된 페이로드를 **합의 레이어**로 반환

페이로드 빌더는 Reth의 블록 생성 과정에서 핵심적인 역할을 하며, MEV 최적화와 효율적인 트랜잭션 선택을 통해 네트워크의 전체적인 성능과 수익성을 향상시킵니다.